Study Notes

Computer Engineering Theory

Compiled August 16, 2026

Contents

1 Digital Logic Fundamentals

1.1 Boolean Algebra

Boolean algebra operates over the set $\{0, 1\}$ under three primary operations: AND ($\cdot$), OR ($+$), and NOT ($^-$). Every digital circuit can be expressed as a combination of these primitives.

De Morgan's Theorems.

$$\overline{A \cdot B} = \overline{A} + \overline{B} \tag{1}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B} \tag{2}$$

These identities let you push a negation through a gate by flipping AND $\leftrightarrow$ OR — invaluable for converting between NAND/NOR implementations.

Note

De Morgan's theorems extend naturally to n variables: $\overline{A_1 \cdot A_2 \cdots A_n} = \overline{A_1} + \overline{A_2} + \cdots + \overline{A_n}$.

1.2 Logic Gates and Truth Tables

The table below shows the four most common two-input gates. NAND and NOR are functionally complete individually — any Boolean function can be realised using only NANDs, or only NORs.

A	B	AND	OR	NAND	NOR
0	0	0	0	1	1
0	1	0	1	1	0
1	0	0	1	1	0
1	1	1	1	0	0

Implementation tip

Prefer NAND-only designs in CMOS: NAND gates are slightly faster and smaller than NOR gates in that technology because PMOS transistors (used in pull-up networks) are inherently slower than NMOS.

1.3 Karnaugh Maps

A Karnaugh map (K-map) is a visual method for minimising Boolean expressions. Groups of 2^n adjacent `1`s correspond directly to product terms in the simplified sum-of-products (SOP) expression.

Grouping rules

1. Groups must be rectangular and contain a power-of-two number of cells ($1, 2, 4, 8 \ldots$).
2. Every `1` must be covered by at least one group.
3. Groups may wrap around edges (the map is toroidal).
4. Prefer the fewest, largest groups to minimise literals.

Don't-care conditions

Don't-care entries (`X`) may be included in a group if doing so makes it larger — but you are not obligated to cover them. Never include an `X` in a group if it does not increase the group's size.

2 Computer Architecture

2.1 Instruction Set Architectures

ISA. An Instruction Set Architecture is the abstract interface between hardware and software: it specifies the programmer-visible registers, data types, addressing modes, and the encoding of every instruction the processor can execute.

The two dominant design philosophies are contrasted below.

Property	RISC	CISC
Instruction count	Many simple instructions	Fewer, complex instructions
Memory access	Load/store only	Any instruction can access memory
Register file	Large (32+)	Small (8 in x86)
Microcode	None (typically)	Extensive
Examples	ARM, RISC-V, MIPS	x86, x86-64

2.2 Pipelining

Pipelining overlaps the execution of multiple instructions by dividing the datapath into discrete stages. A classic five-stage RISC pipeline:

1. IF — Instruction Fetch: read instruction from memory at PC.
2. ID — Instruction Decode / Register Read.
3. EX — Execute: ALU operation or address calculation.
4. MEM — Memory access (loads and stores only).
5. WB — Write Back: result to register file.

Pipeline hazards

- Structural — two instructions require the same hardware resource simultaneously.
- Data — an instruction depends on a result not yet written back (RAW, WAR, WAW).
- Control — branch outcome is unknown; fetched instructions may be wrong.

Forwarding / bypassing

Most RAW data hazards are resolved by forwarding: routing the output of the EX or MEM stage directly back to the EX stage input, without waiting for WB. This eliminates the majority of pipeline stalls without compiler intervention.

2.2.1 Branch prediction

Static prediction (always-taken / always-not-taken) achieves roughly 60% accuracy. Dynamic predictors maintain a history table; a 2-bit saturating counter scheme typically reaches 85–90% accuracy on real workloads.

Loop unrolling

Loop unrolling reduces branch overhead and exposes more instruction-level parallelism to the scheduler — useful when the loop body is small and the iteration count is known at compile time.

3 Operating Systems

3.1 Process Scheduling

Scheduling criteria. Key metrics: CPU utilisation, throughput (processes/unit time), turnaround time (submit $\rightarrow$ complete), waiting time (time in ready queue), and response time (submit $\rightarrow$ first response).

3.1.1 Completely Fair Scheduler (CFS)

Linux's default scheduler assigns a virtual runtime (`vruntime`) to each task. The task with the smallest `vruntime` is always scheduled next; it accumulates `vruntime` at a rate inversely proportional to its nice weight. Tasks are stored in a red-black tree keyed on `vruntime`, giving $O(\log n)$ scheduling decisions.

Inspecting CFS scheduling weights (Linux)

```
# View the scheduling policy and priority of a process
chrt -p <PID>

# Inspect vruntime and load weight via /proc
cat /proc/<PID>/sched

# Change the nice value of a running process
renice -n 5 -p <PID>
```

3.2 Memory Management

3.2.1 Virtual memory and paging

Each process sees a private, contiguous virtual address space. The MMU translates virtual addresses to physical addresses via a page table; modern systems use multi-level tables (e.g. four levels on x86-64) to avoid storing entries for the entire virtual space.

> Translation Lookaside Buffer (TLB). A small, fully-associative cache inside the MMU storing recent virtual-to-physical mappings. A TLB hit resolves a translation in 1–2 cycles; a miss requires a page-table walk costing 10–100+ cycles.

TLB shootdowns in SMP

When a mapping is modified on one core, all other cores must invalidate their TLB entries for that page. This inter-processor interrupt (IPI) broadcast — the TLB shootdown — is a significant scalability bottleneck on many-core systems under high `munmap` / `mmap` rates.

3.2.2 Page replacement policies

Algorithm	Description	Practical cost
OPT	Evict the page used furthest in the future	Impossible online
LRU	Evict least recently used page	High (full timestamp)
Clock	Approximates LRU with a reference bit	Low
LFU	Evict least frequently used page	Medium
Linux (2Q)	Active/inactive list pair with promotion	Low

4 Data Structures and Algorithms

4.1 Asymptotic Complexity

Big-O notation. $f(n) = O(g(n))$ iff $\exists\, c > 0,\ n_0 > 0$ such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$. It describes an asymptotic upper bound — it says nothing about constant factors or small inputs.

Note

O, Ω, and Θ are distinct. When people say "the complexity is $O(n \log n)$" they usually mean $\Theta(n \log n)$ — both upper and lower bounded. Sloppy usage propagates through a lot of textbooks; be precise in your own work.

4.2 Sorting algorithms — complexity summary

Algorithm	Best	Average	Worst	Space
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

4.3 Example: merge sort

Merge sort — Python

```python
def merge_sort(arr: list[int]) -> list[int]:
    """Recursive merge sort. Returns a new sorted list."""
    if len(arr) <= 1:
        return arr

    mid   = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return _merge(left, right)


def _merge(left: list[int], right: list[int]) -> list[int]:
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
```

```
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]
```

When to use Timsort

Python's built-in `sorted()` and `list.sort()` both use Timsort, which exploits existing runs of sorted data in real inputs. For nearly-sorted sequences it degrades gracefully to $O(n)$. Prefer it over rolling your own sort for anything outside embedded or GPU contexts.

4.4 Graph representations

Two standard representations for a graph $G = (V, E)$:

- Adjacency matrix

An $|V| \times |V|$ matrix where entry $(i, j) = 1$ iff edge $(i, j) \in E$. Space: $O(V^2)$. Edge lookup: $O(1)$. Preferred for dense graphs.

- Adjacency list

An array of $|V|$ lists, each containing the neighbours of vertex i. Space: $O(V + E)$. Edge lookup: $O(\deg(v))$. Preferred for sparse graphs — which is nearly every real-world network.

BFS — C

```c
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

#define MAX 128

void bfs(int adj[MAX][MAX], int n, int src) {
    bool visited[MAX] = {false};
    int  queue[MAX], front = 0, rear = 0;

    visited[src]  = true;
    queue[rear++] = src;

    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);
        for (int u = 0; u < n; u++) {
            if (adj[v][u] && !visited[u]) {
                visited[u]    = true;
                queue[rear++] = u;
            }
```

```
            }
        }
    }
```

5 Networking

5.1 The TCP/IP Stack

#	Layer	Responsibility	Key protocols
4	Application	User-facing services	HTTP, DNS, SMTP, SSH
3	Transport	End-to-end delivery, flow control	TCP, UDP, QUIC
2	Internet	Logical addressing, routing	IP (v4/v6), ICMP
1	Link	Framing, MAC addressing	Ethernet, Wi-Fi

5.2 TCP three-way handshake

1. Client sends `SYN` (seq = x).
2. Server replies `SYN-ACK` (seq = y, ack = $x + 1$).
3. Client sends `ACK` (ack = $y + 1$). Connection established.

SYN flood attacks

A SYN flood exploits the half-open connection state: an attacker sends many `SYN` packets with spoofed source IPs, exhausting the server's connection backlog. Mitigated with SYN cookies: the server encodes state into the `ISN` instead of allocating a TCB, deferring allocation until the `ACK` is verified.

Note

QUIC (RFC 9000) eliminates the TCP handshake latency for repeat connections via 0-RTT resumption and integrates TLS 1.3 natively — the entire handshake costs 1 RTT on first connection, 0 RTT on resumption.